Penetration Testing Lab Report

Web Application Unrestricted File Upload and Remote Code Execution (RCE) Vulnerability

Project Title: Arbitrary File Upload Investigation & Server Compromise

Target Application: StaffHub (localhost:8080)

Author / Tester: Sudip Bastola

Assessment Date: August 14, 2026

Assessment Type: Controlled Academic / Lab Web Application Security Assessment

1. Vulnerability Information

- **Vulnerability Name:** Unrestricted File Upload leading to Remote Code Execution (RCE)
- **CWE ID:** CWE-434: Unrestricted Upload of File with Dangerous Type
- **OWASP Category:** OWASP Top 10:2021 – A04: Insecure Design / A03: Injection
- **Severity:** Critical
- **Severity Justification:** Allowed an authenticated portal user to bypass client-side file-type restrictions on the profile photo upload functionality, upload an arbitrary PHP web shell directly into an accessible web-root directory (/uploads/), and execute arbitrary operating-system commands with web-server privileges (www-data), enabling total filesystem enumeration and unauthorized flag file discovery.
- **Location:** http://localhost:8080/myprofile.php
- **Vulnerable Parameter:** photo (multipart/form-data POST body parameter)

2. Description

Unrestricted File Upload occurs when a web application allows users to upload files to the server filesystem without adequately validating, sanitizing, or restricting the file's type, extension, content, or storage location. If an application permits executable file extensions (such as .php) to be stored within an executable web-accessible directory, an attacker can directly request the uploaded script via HTTP to execute arbitrary code within the context of the hosting web server process.

During the security assessment of the StaffHub web application, the profile photo update feature located at http://localhost:8080/myprofile.php was analyzed. Although the user interface indicated support only for image formats (JPG, PNG, GIF), testing revealed that the backend failed to enforce server-side file extension and MIME type validation. By intercepting the file upload HTTP POST request with Burp Suite, the uploaded filename was modified from an image extension to a PHP script (test image.php) containing a lightweight web shell (<?php system($_GET['cmd']); ?>). The server accepted the file, stored it in the publicly accessible /uploads/ directory, and returned a success message. Direct HTTP requests to http://localhost:8080/uploads/test%20image.php allowed arbitrary OS command execution under the www-data service user context, enabling complete server enumeration and file retrieval.

3. Tools Used

- **Brave Browser:** Used to navigate the StaffHub employee portal, access the profile management interface on myprofile.php, submit file upload requests, and invoke the uploaded PHP web shell to execute shell commands and display terminal outputs.
- **Burp Suite Community Edition (v2026.7.3):** Used as an HTTP interception proxy to intercept, inspect, modify, and replay multipart/form-data file upload requests, enabling the tampering of filenames and insertion of PHP executable payloads via the Proxy Intercept and Repeater modules.

4. Step-by-Step Exploitation

Step 1 - Baseline Profile Photo Upload Testing

Action: Navigated to http://localhost:8080/myprofile.php and uploaded a standard image file (test image.jpg) through the 'Update Profile Photo' form.

Reason: To establish baseline application behavior for legitimate image file uploads and observe server handling and response messages.

Parameter: http://localhost:8080/myprofile.php

File: Submitted legitimate JPEG file named test image.jpg.

Observation : The application successfully accepted and processed the image, returning a green success notification: 'Profile photo uploaded successfully: test image.jpg'.

Security Significance: Confirms the existence of functional file upload handling on the profile endpoint and establishes the expected UI feedback pattern.

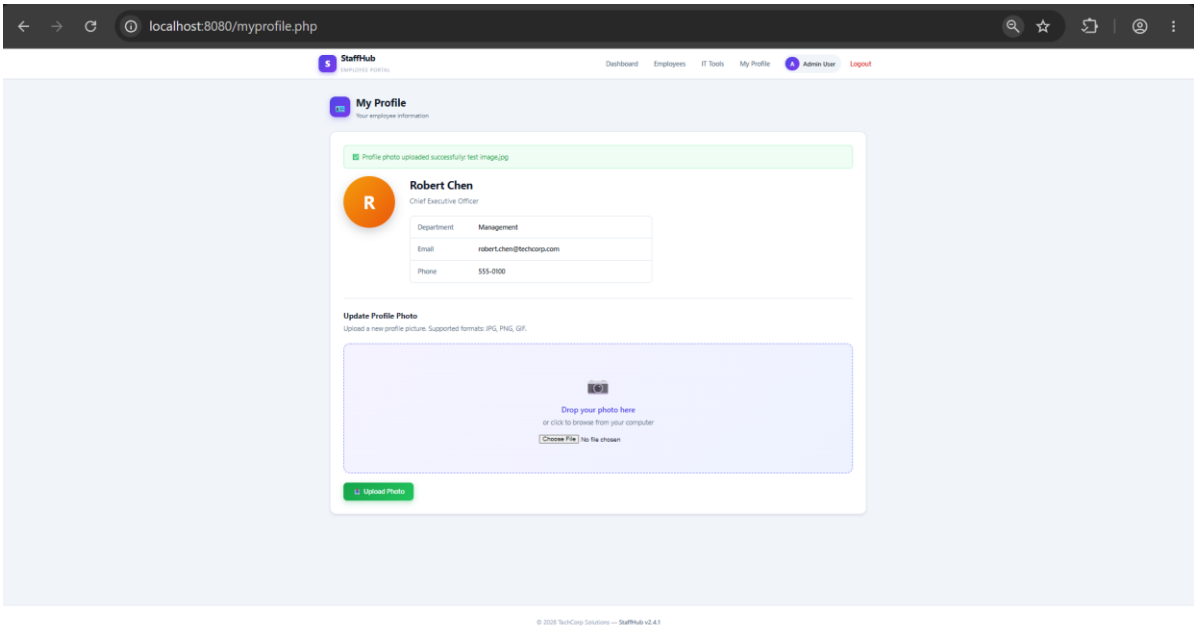

Figure 1 StaffHub My Profile Page Showing Successful Baseline Upload of test image.jpg

Step 2 - Intercepting Legitimate Upload Request in Burp Suite

Action: Enabled Burp Suite Proxy Intercept and captured the raw HTTP POST request generated when uploading test image.jpg to /myprofile.php.

Reason: To analyze the structure of the multipart/form-data request, inspect headers, identify parameters, and assess how file metadata is transmitted to the backend.

URL : POST http://localhost:8080/myprofile.php

Observation : The request transmits the file via the photo parameter with a client-supplied filename (test image.jpg) and Content-Type: image/jpeg header.

Security Significance: Reveals the exact multipart structure required to construct and manipulate file upload payloads against the backend server.

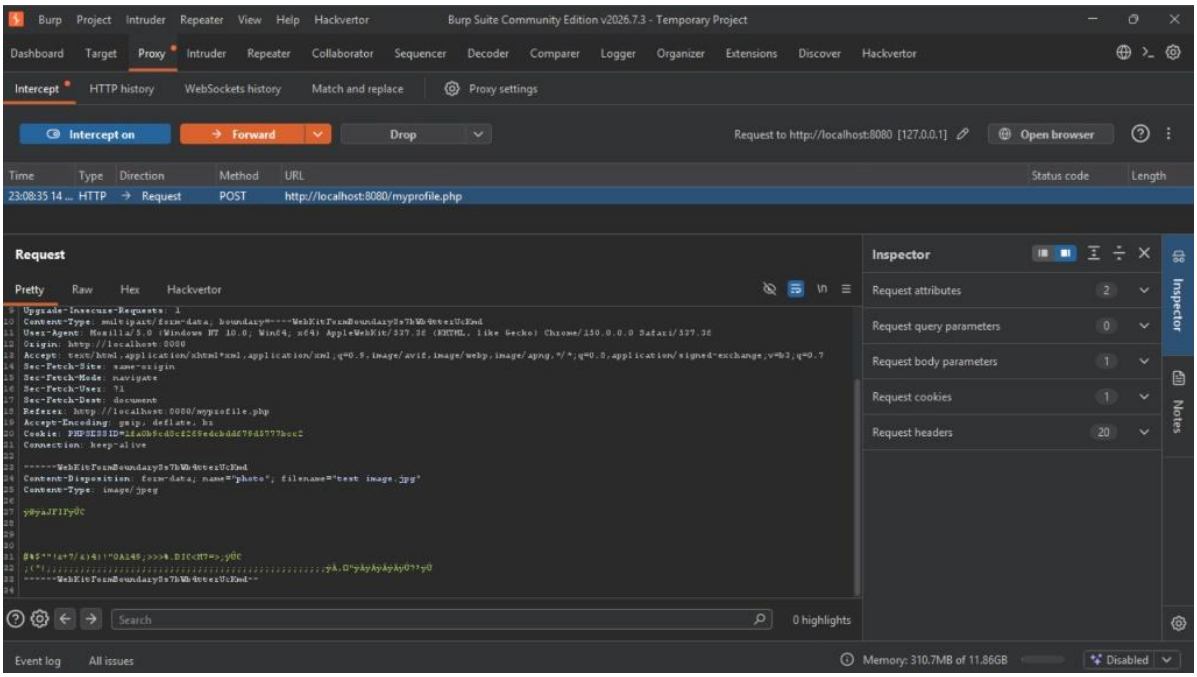

Figure 2 Burp Suite Proxy Intercepting the Multipart Upload Request for test image.jpg

Step 3 - Crafting and Testing Executable PHP Payload via Burp Repeater

Action: Sent the captured upload request to Burp Suite Repeater, altered the filename parameter from test image.jpg to test image.php, maintained the Content-Type: image/jpeg header, and replaced the binary image body with a single-line PHP command execution payload.

Reason: To test whether the server validates the file extension against an allowlist on the backend, or if it naively trusts the client-supplied Content-Type header or permits executable PHP extensions.

Parameter: POST http://localhost:8080/myprofile.php | Parameter: photo

Payload : Content-Disposition: form-data; name="photo"; filename="test image.php" Content-Type:image/jpeg
<?php system($_GET['cmd']); ?>

Explanation: <?php system($_GET['cmd']); ?> is a minimal PHP web shell that receives an operating system command passed via the 'cmd' GET parameter and executes it directly on the server shell using the native system() function.

Observation : The server returned an HTTP/1.1 200 OK response, confirming that the modified upload request was processed without being blocked by server-side validation filters.

Security Significance: Indicates a critical lack of extension filtering, allowing arbitrary PHP executable code to be accepted by the server.

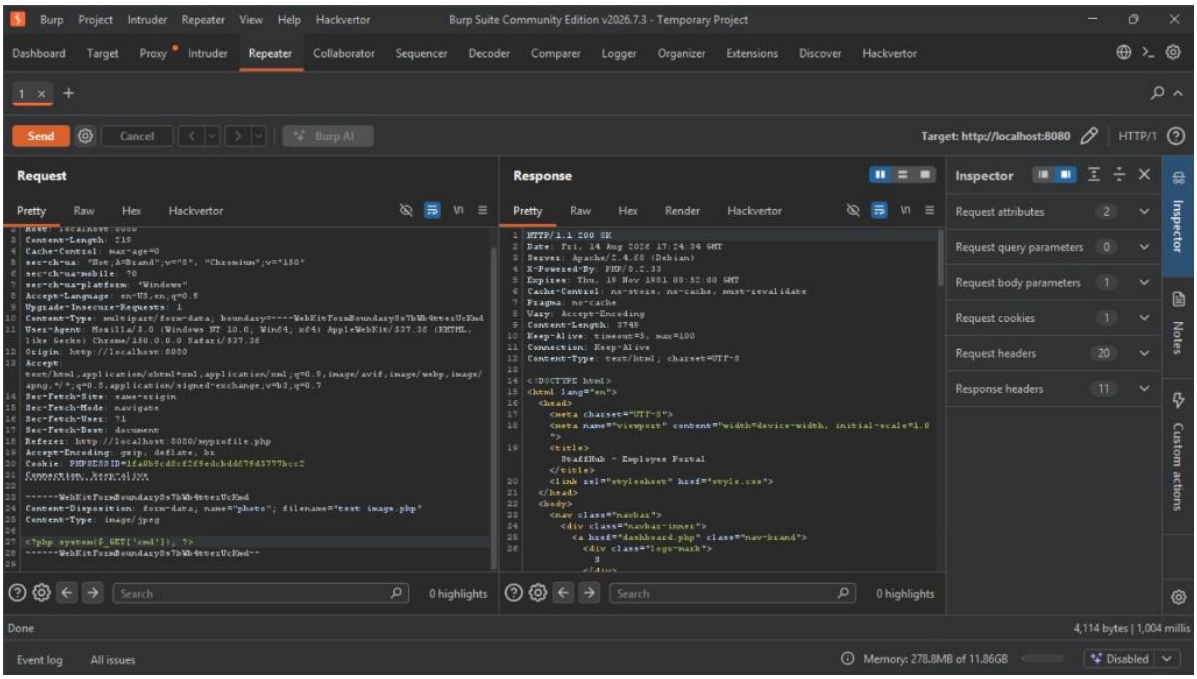

Figure 3 Burp Suite Repeater Showing Successful Upload Response for test image.php Web Shell Payload

Step 4 - Verifying Web Shell Upload Confirmation in Application UI

Action: Forwarded the manipulated upload request through Burp Suite Proxy and observed the resulting rendered page in the Brave Browser.

Reason: To confirm that the application backend committed the PHP file to the filesystem and verify the confirmation status displayed to the user.

URL : http://localhost:8080/myprofile.php

Observation : The web application rendered an alert banner stating: 'Profile photo uploaded successfully: test image.php'.

Security Significance: Definitively confirms that the PHP script was accepted and saved under its original filename on the server.

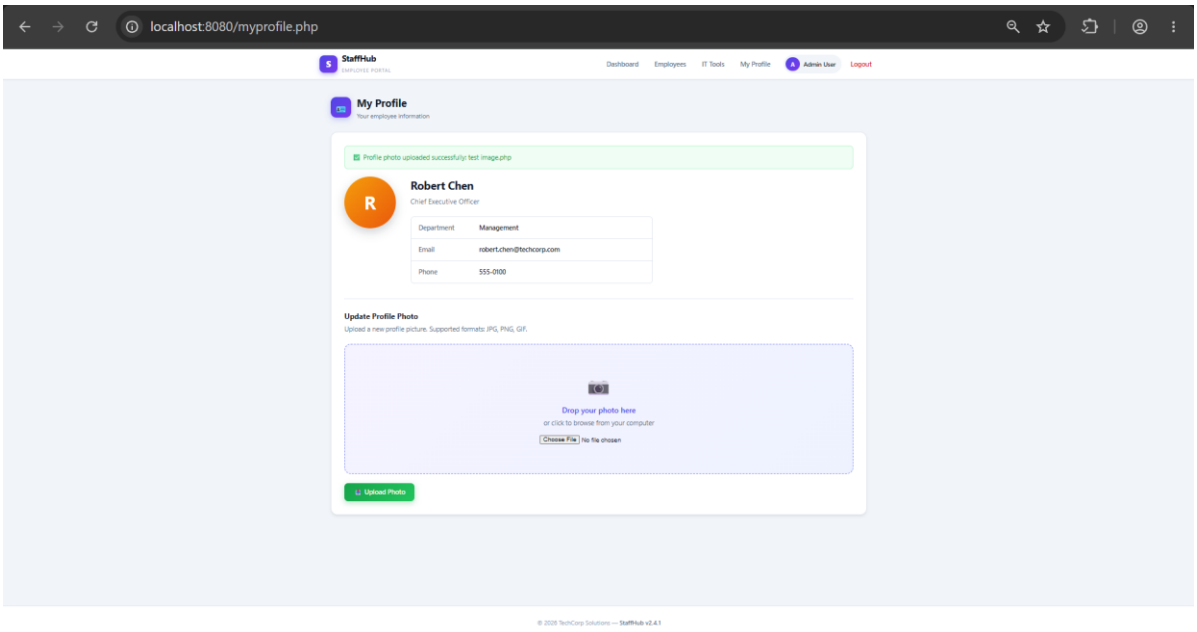

Figure 4 StaffHub UI Conforming Successful Upload of test image.php

Step 5 - Intercepting Direct PHP Script Upload via Proxy

Action: Enabled Burp Suite Proxy Intercept initiated a new profile photo upload in the browser, and replaced the legitimate file body and filename in the intercepted HTTP request with the verified PHP web shell payload copied from Burp Repeater before forwarding the request to the server.

Reason: To execute the actual file upload transaction through the client's live browsing session, ensuring the web shell is written to the web server's storage and accessible within the active application context.

URL /: POST http://localhost:8080/myprofile.php

Raw Intercepted Payload: Content-Disposition: form-data; name="photo"; filename="testimage.php"
Content-Type:image/jpeg
<?php system($_GET['cmd']); ?>

Observation : The modified multipart request was forwarded cleanly through the proxy to the backend without triggering client-side rejection or connection errors.

Security Significance: Demonstrates that client-side file-type restrictions can be bypassed by modifying intercepted HTTP traffic in transit, allowing arbitrary executable code to be delivered directly to the server backend.

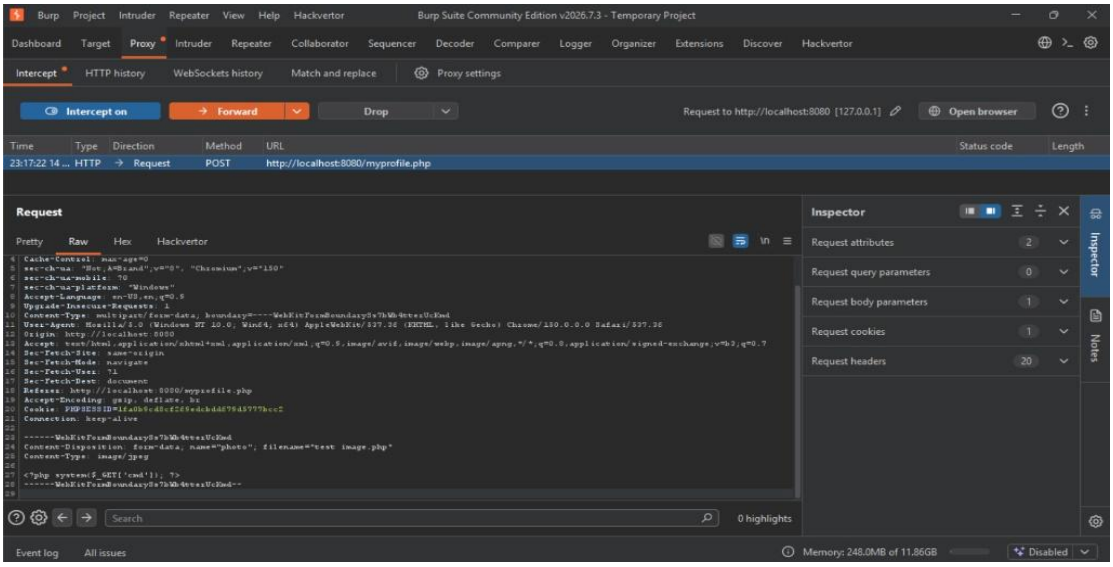

Figure 5 Burp Suite Proxy Intercept View Showing Replaced PHP Payload Forwarded to the Server

Step 6 - Remote Code Execution & User Context Identification

Action: Navigated to the uploaded script URL at http://localhost:8080/uploads/test%20image.php in the brave browser and passed the whoami command via the cmd GET parameter.

Reason: To verify that the uploaded PHP script is executable by the web server engine and determine the operating system user account under which the code executes.

URL : http://localhost:8080/uploads/test%20image.php?cmd=whoami

Command: whoami

Explanation: The whoami command outputs the username of the current effective user running the active process in Unix-like operating systems.

Observation / Result: The browser immediately returned cleartext output: www-data.

Security Significance: Confirms full Remote Code Execution (RCE) on the server, operating under the privileges of the Apache/web server daemon user (www-data).

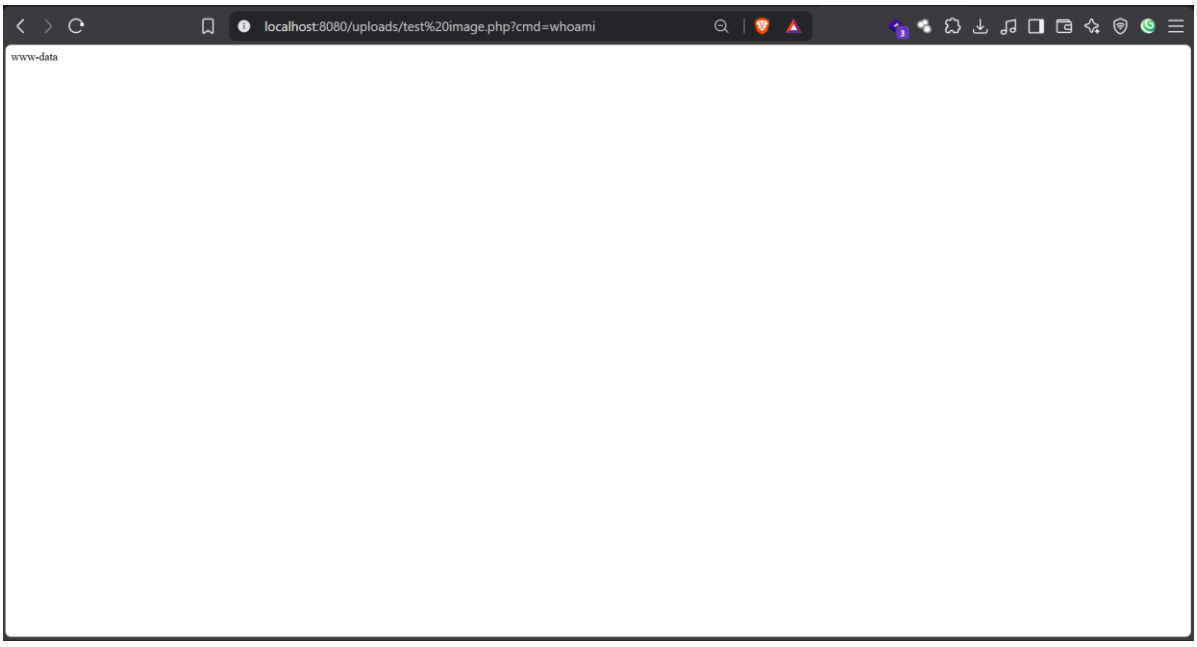

Figure 6 Remote Code Execution via Uploaded Web Shell Returning User Identity (www-data)

Step 7 - Server Working Directory Enumeration

Action: Executed the pwd command by browsing to http://localhost:8080/uploads/test%20image.php?cmd=pwd.

Reason: To identify the absolute filesystem directory path where uploaded files are stored on the server.

URL : http://localhost:8080/uploads/test%20image.php?cmd=pwd

Command: pwd

Explanation: The pwd (print working directory) command outputs the current absolute path of the executing process.

Observation / Result: The browser returned the path: /var/www/html/uploads.

Security Significance: Confirms the web server root architecture (/var/www/html/) and verifies that the uploads directory is directly accessible within the public web tree without script execution restrictions.

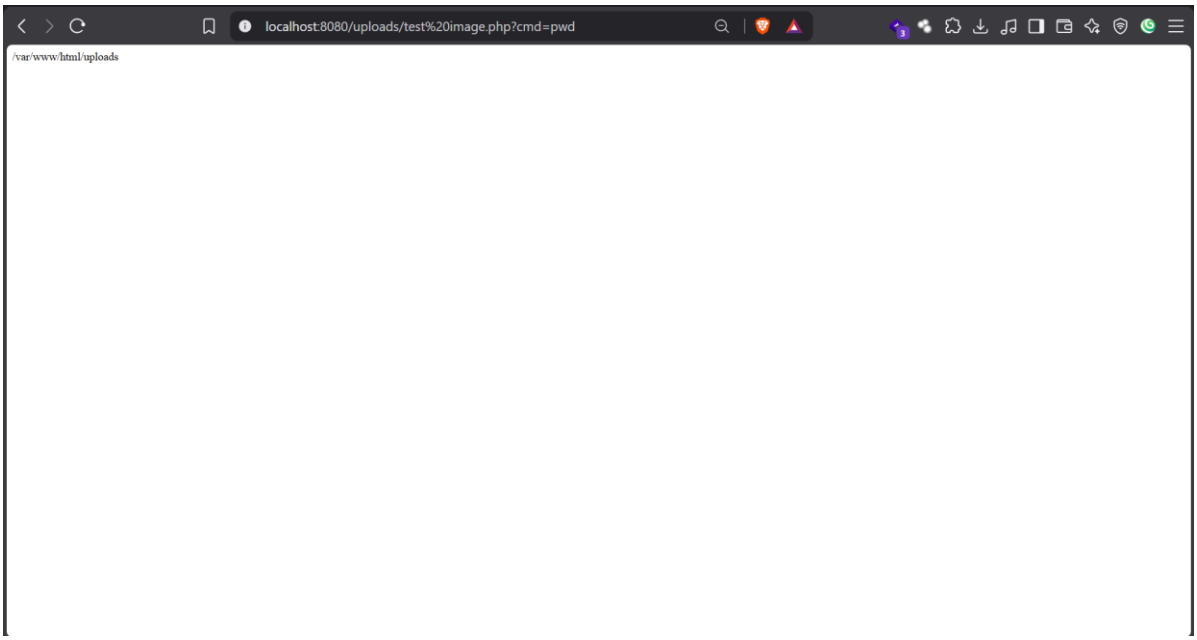

Figure 7 Web Shell Command Execution Output Displaying Current Working Directory (/var/www/html/uploads)

Step 8 - Filesystem Search and Flag Discovery

Action: Executed a filesystem search command across the web directory tree using the find utility via http://localhost:8080/uploads/test%20image.php?cmd=find%20/var/www%20-name%20%22*flag*%22.

Reason: To locate restricted flag files and assessment artifacts distributed across the web server filesystem.

URL : http://localhost:8080/uploads/test%20image.php?cmd=find%20/var/www%20-name%20%22*flag*%22

Command: find /var/www -name "*flag*"

Explanation: The find command searches the /var/www directory structure for any files or folders whose names match the pattern *flag*.

Observation / Result: The server returned the absolute paths of all internal flag files: /var/www/flags/flag4.txt, /var/www/flags/flag2.txt, /var/www/flags/flag1.txt, and /var/www/flags/flag3.txt.

Security Significance: Demonstrates arbitrary filesystem visibility and uncovers the exact path to the target Flag 4 file (/var/www/flags/flag4.txt), fully validating the impact of the unrestricted upload and remote code execution vulnerability.

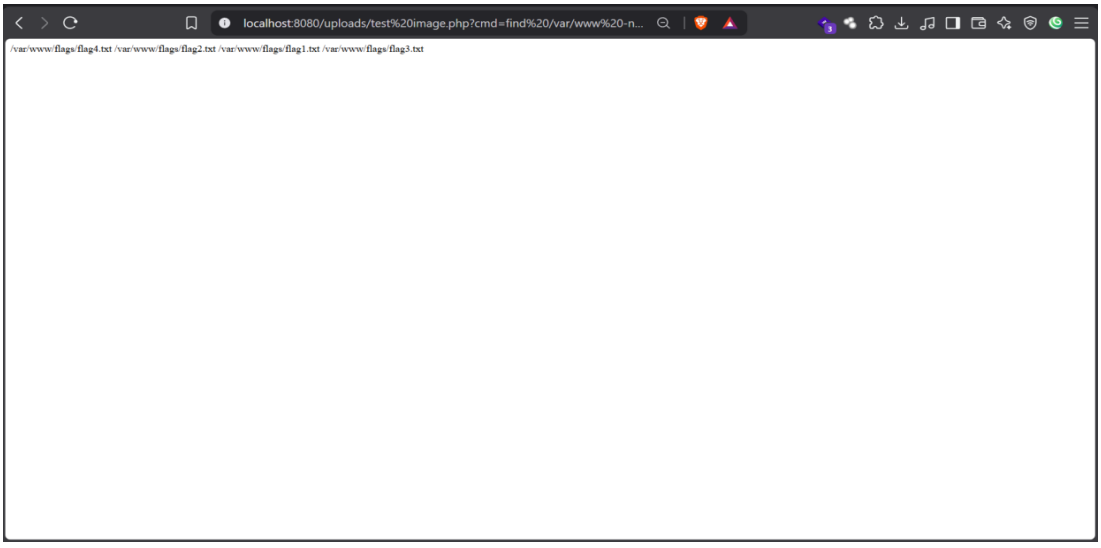

Figure 8 Filesystem Search via find Command Revealing Internal Flag File Locations Including Flag 4

5. Proof of Exploitation

The evidence gathered during the assessment confirms that the profile photo upload endpoint on /myprofile.php fails to validate file extensions on the server side. An attacker can upload an executable PHP script into the web-accessible /uploads/ directory and execute arbitrary operating-system commands with web-server privileges (www-data). Through sequential execution of whoami, pwd, and find commands, unrestricted command execution and discovery of server-side flag files were fully demonstrated.

Captured Flag: FLAG{Th1s_1s_N0t_Th3_Ph0t0_U_R_L00k1ng_4}

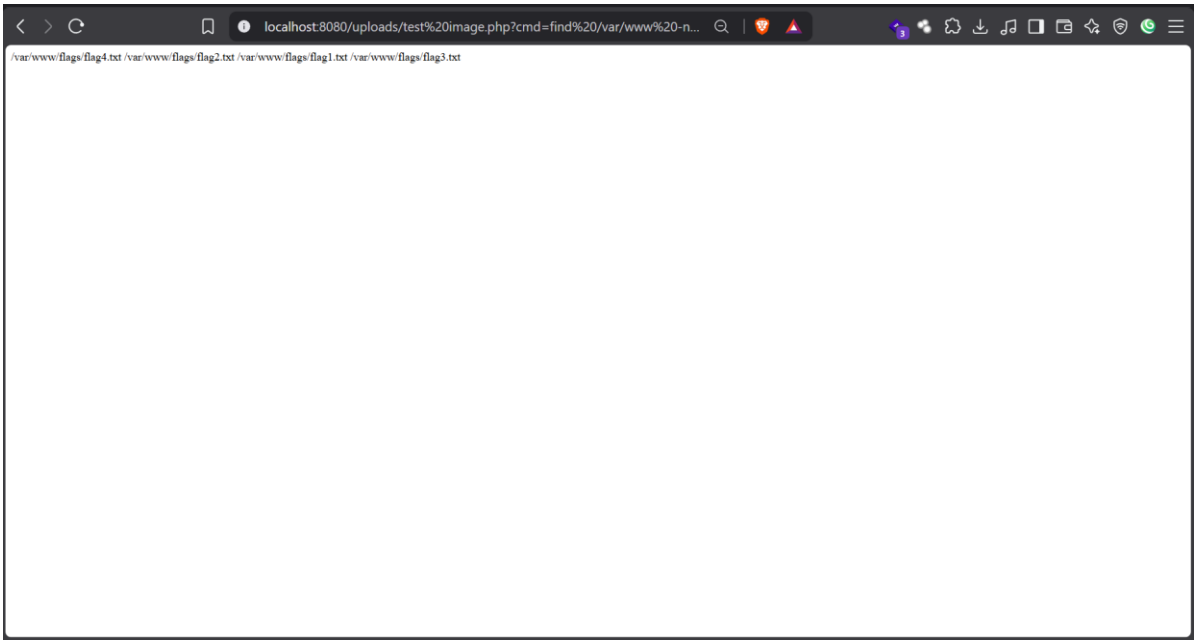

Figure 9 Proof of Arbitrary Command Execution and Flag File Enumeration via Uploaded Web Shell

6. Business Impact

Demonstrated in the Lab

During testing on the StaffHub application, bypassing client-side file upload restrictions allowed the deployment of an interactive PHP web shell (test image.php) directly into the /uploads/ web directory. Through HTTP GET parameters, arbitrary operating system commands were executed under the www-data account context. This enabled full identification of the web server root (/var/www/html/uploads) and comprehensive enumeration of internal directories, uncovering sensitive system artifacts including /var/www/flags/flag4.txt.

Potential Real-World Impact

In a production enterprise deployment, an unmitigated Unrestricted File Upload vulnerability leading to Remote Code Execution represents a catastrophic security failure. An attacker gaining shell access under the web-server user can read proprietary source code, access database credentials, extract confidential employee and corporate records, alter web assets, or plant persistent backdoors. Furthermore, the attacker could leverage local privilege escalation exploits to gain root privileges, pivot laterally across internal network segments, compromise connected database and authentication servers, or deploy ransomware, resulting in massive data theft, complete service disruption, regulatory non-compliance, and severe reputational damage.

7. Remediation Recommendations

To remediate this Unrestricted File Upload and Remote Code Execution vulnerability, the following server-side security controls should be implemented:

- **Strict Server-Side Extension Allowlisting:** Enforce strict server-side validation against an immutable allowlist of safe image extensions (e.g., .jpg, .jpeg, .png, .gif). Reject all files with executable extensions (e.g., .php, .phtml, .php5, .phar, .cgi) regardless of the client-supplied Content-Type header.
- **File Content & MIME Type Verification:** Inspect file headers on the server-side using robust libraries to ensure that the uploaded payload matches genuine image data structures rather than script text.
- **File Renaming & Randomization:** Do not preserve user-supplied filenames on disk. Automatically generate a cryptographically random, alphanumeric filename combined with a strictly enforced, sanitized extension upon saving.
- **Disable Script Execution in Upload Directories:** Configure the web server to explicitly disable PHP and script execution within the /uploads/ directory.
- **Store Uploads Outside the Web Root:** Store user-uploaded media in a dedicated directory located entirely outside the public web server.
- **Principle of Least Privilege:** Ensure the web server process (www-data) runs with minimal operating system privileges and has write access strictly restricted to designated non-executable storage locations.
- **Security Logging and Monitoring:** Implement real-time file upload logging and integrity monitoring to immediately detect and alert on anomalous upload patterns, unexpected file extensions, or web shell signatures.

References

Common Weakness Enumeration, n.d. *Unrestricted Upload of File with Dangerous Type.* [Online]
Available at: https://cwe.mitre.org/data/definitions/434.html
[Accessed 14 08 2026].

OWASP Top 10:2021, n.d. *A04:2021 – Insecure Design.* [Online]
Available at: https://owasp.org/Top10/2021/A04_2021-Insecure_Design/
[Accessed 14 08 2026].

OWASP, n.d. *File Upload Cheat Sheet.* [Online]
Available at: https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html
[Accessed 14 08 2026].

OWASP, n.d. *Unrestricted File Upload.* [Online]
Available at: https://owasp.org/www-community/vulnerabilities/Unrestricted_File_Upload
[Accessed 14 08 2026].

PortSwigger, n.d. *File upload vulnerabilities.* [Online]
Available at: https://portswigger.net/web-security/file-upload
[Accessed 14 08 2026].